

Segmentation

● FCN세그멘테이션 미리 알기

완전 합성곱 네트워크(Fully Convolutional Networks) 정의

이미지의 각 픽셀에 클래스 레이블을 할당하는 컴퓨터 비전 기술인

시맨틱 세그멘테이션(Semantic Segmentation)을 위한 딥러닝 모델

이미지 → 픽셀마다 클래스 예측 하는 네트워크. 기존 CNN은 "하나의 이미지 → 하나의 라벨",

FCN은 "하나의 이미지 → 픽셀마다 라벨(=마스크)"

- X데이터 = 이미지파일
- Y 데이터 = "픽셀마다 클래스 정답(정답 마스크)" 로서 label index

"이 픽셀은 배경(인덱스 0)"

"이 픽셀은 사람(인덱스 1)"

"이 픽셀은 고양이(인덱스 2)"

X: [B, 3, H, W]

Y: [B, H, W] # 채널없음

0 0 0 0 0 0 0	• 0 = 배경
0 1 1 1 0 0 0	• 1 = 사람
0 1 2 1 0 0 0	• 2 = 고양이
0 1 1 1 0 0 0	

각 픽셀단위로 클래스 예측

Pred: [B, num_classes, H, W]

X (컬러 이미지)

→ FCN → Pred (픽셀별 확률맵)

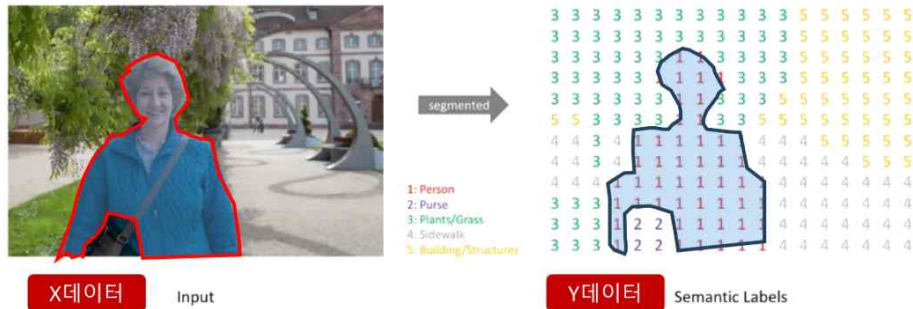
→ argmax → 예측 마스크

→ Y(정답 마스크)와 비교하며 Loss 계산

1 이미지세그멘테이션의 개요

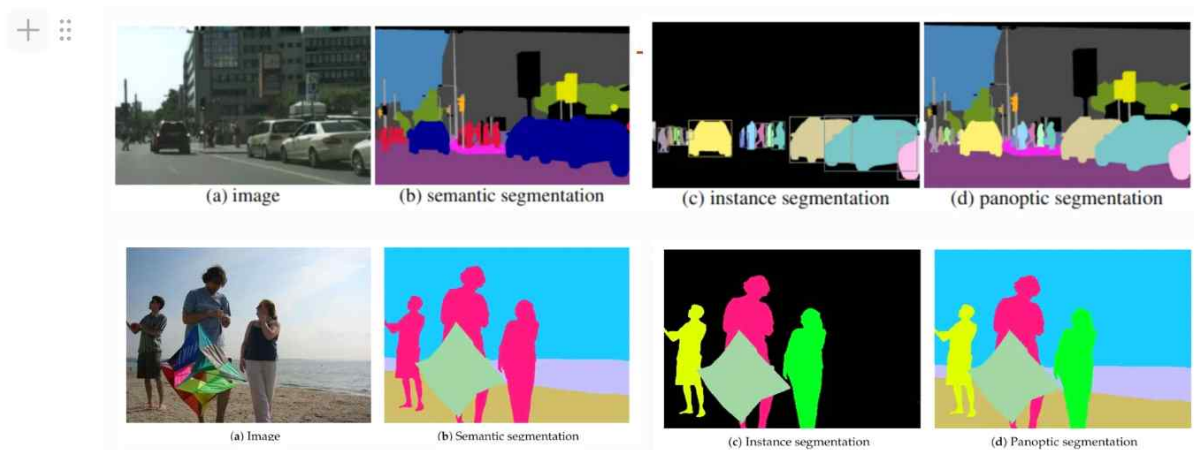
(1) 세그멘테이션의 정의

이미지를 픽셀 단위로 분석하여, 각 픽셀이 특정 클래스(예: 사람, 배경 등)에 속하는지 또는 특정 객체(예: 자동차 #1, 자동차 #2 등)에 속하는지를 식별하는 작업.



(2) Segmentation의 종류

- Semantic Segmentation: "이 이미지는 차량과 도로로 이루어져 있다."
- Instance Segmentation: "여기에는 차량 3대가 각각 다른 위치에 있다."
- Panoptic Segmentation: "차량 A, 차량 B, 도로는 각각 어디에 위치해 있는가?"



구분	설명	특징	예시
Semantic Segmentation	픽셀마다 클래스 레이블을 할당	같은 클래스에 속한 모든 픽셀을 하나로 처리	"모든 자동차는 같은 클래스로 묶임."
Instance Segmentation	픽셀마다 클래스 레이블뿐만 아니라 개별 객체의 구분도 포함	같은 클래스 내에서도 개체 간의 구분 가능	"자동차 1, 자동차 2를 별도로 구분."

(3) 세그멘테이션의 필요성

- **이미지 분석의 정밀화:**
 - 이미지 분류(Classification)는 전체 이미지에 대해 하나의 레이블만 예측.
 - 객체 탐지(Object Detection)는 바운딩 박스로 객체를 탐지하지만, 물체의 정확한 경계는 모호.
 - 세그멘테이션은 픽셀 수준의 정보를 활용하여 더욱 정밀한 분석 제공.
 - **주요 응용 분야:**
 1. **자율주행:** 도로의 차선, 보행자, 신호등 등 다양한 객체를 픽셀 수준에서 이해.
 2. **의료 영상 처리:** CT, MRI, X-Ray 이미지에서 종양, 장기 등의 정확한 윤곽 탐지.
 3. **위성 영상 분석:** 토지 이용 유형, 건물, 도로 등 지리적 객체 식별.
 4. **AR/VR:** 가상 객체와 실제 환경 간의 정밀한 상호작용 지원.
 - **데이터 활용의 효율성:**
 - 픽셀 단위로 정보를 제공함으로써 AI 모델의 이해도를 높이고, 더욱 풍부한 분석 결과 제공.
 - 객체의 경계, 크기, 모양 등 상세한 정보를 제공하여 다양한 실무 문제 해결 가능.
-

(4) 세그멘테이션의 어려움

- **픽셀 단위의 복잡도:** 고해상도 이미지에서는 수백만 개의 픽셀을 처리해야 함.
 - **클래스 불균형 문제:**
 - 객체와 배경의 비율이 크게 차이 나는 경우가 많음.
 - 예: 도로에서 차선은 픽셀이 적고 배경은 많음.
 - **라벨링 비용:** 픽셀 단위로 레이블링하는 작업은 비용이 높고 시간 소요가 큼.
 - **모델의 일반화:** 다양한 환경에서 일관된 성능을 유지하는 것이 도전 과제.
-

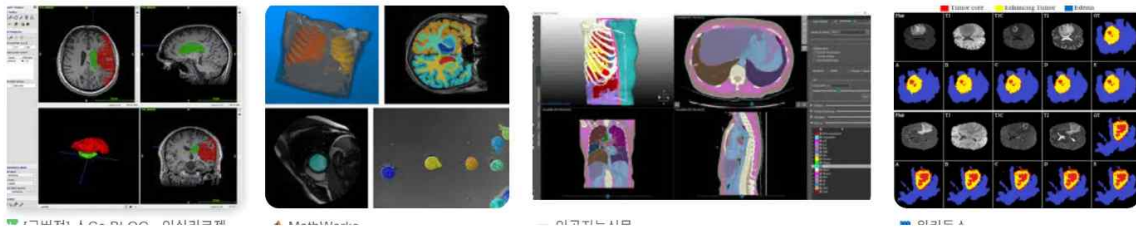
(5) 세그멘테이션의 진화

- **과거:**
 - 전통적 방법(예: SIFT, SURF)으로 경계 탐지 및 세그멘테이션 수행.
 - 낮은 정확도와 제한된 활용 가능성.
 - **현대:**
 - 딥러닝의 등장으로 세그멘테이션 정확도와 속도 대폭 향상.
 - CNN 기반의 Fully Convolutional Networks(FCN), U-Net 등이 선두 주자로 자리 잡음.
-

2 주요 응용 분야

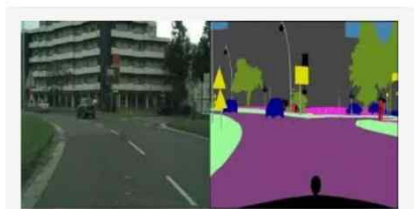
(1) 의료 영상 분석

- **응용 분야:** CT, MRI, 초음파 등 의료 영상에서 특정 장기, 병변(종양 등)을 픽셀 단위로 분리.
- **목표:** 병변의 정확한 위치와 크기를 측정해 진단 보조.
- **대표 사례:**
 - **간 종양 탐지:** 간 CT 이미지를 분석하여 종양의 경계를 세그멘테이션.
 - **심장 구조 분석:** 심장의 심실과 심벽을 분리해 심혈관 질환 연구.

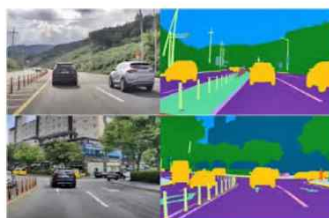


(2) 자율 주행

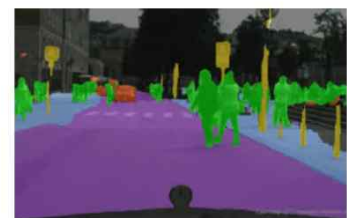
- **응용 분야:** 도로 위 객체(차량, 보행자, 도로, 신호등)와 배경을 분리.
- **목표:** 차량과 보행자의 움직임 예측, 충돌 방지.
- **대표 사례:**
 - **Semantic Segmentation:** 도로, 차선, 배경 구분.
 - **Instance Segmentation:** 차량, 보행자, 자전거를 개별 객체로 구분.



Medium
DNN] Segmentation_00 자율주행자동차를 위한 seg...



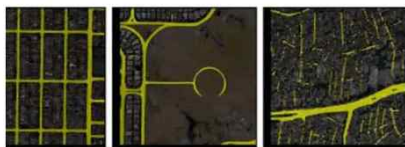
매일경제
車도 탈선하면 안돼요"...스트라드비전, 자율주행 ...



NVIDIA 블로그
자동차에 눈을 달다: 자율주행자동차는 어떻게 주...

(3) 위성 및 항공 영상 분석

- **응용 분야:** 위성 이미지에서 건물, 강, 도로, 농지 등의 픽셀 분리.
- **목표:** 도시 계획, 재난 관리, 환경 보호.
- **대표 사례:**
 - 건물의 위치와 크기를 세그멘테이션하여 건축밀도를 분석.
 - 홍수로 인한 침수 지역 탐지.



GitHub
GitHub - CHEOLLIAN/Satellite-image-object-segmen...



인공지능신문
향우연, AI 위성영상 분석 기술 우수성 입증...



Deep Block
토지 관리를 혁신하는 AI 플랫폼 ...



Naver Blog
AI 기반 위성 영상 분석은 어떤...

(4) 증강 현실 (AR) 및 가상 현실 (VR)

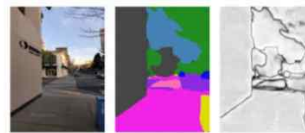
- **응용 분야:** 객체와 배경을 분리하여 가상 객체와 상호작용.
- **목표:** 실시간 객체 추적 및 증강 현실 콘텐츠 렌더링.
- **대표 사례:** AR 기반 가구 배치 시 배경과 가구 객체를 분리.



Samsung SDS
생산성을 높이는 증강현실 기술 증강현실 ...



하이퍼클라우드
하이퍼클라우드 TECH



Google for Developers
Unity의 AR Foundation에 대한 사용자 환경 이해 | AR...



데일리메디
의료계도 비대면·가상현실 넘어 앞으...

(5) 영상 편집 및 생성

- **응용 분야:** 이미지나 비디오에서 객체를 분리해 조작.
- **목표:** 배경 제거, 객체 이동, 재구성.
- **대표 사례:** 배경을 투명화하거나 다른 배경으로 대체.



Original



Segmentation Mask



Background



Combined Result

3 주요 데이터셋 소개

(1) COCO (Common Objects in Context)

<https://cocodataset.org/#download>

- 특징:

- 80개 클래스, 약 20만 개 이미지, 약 150만 개의 객체 어노테이션.
- Semantic Segmentation과 Instance Segmentation 모두 지원.

- 적용 사례:

- 일상적인 객체(사람, 자동차, 동물 등) 인식 및 분리.
- 리스트

```
# 1) COCO annotation 다운로드 (세그멘테이션 포함)
```

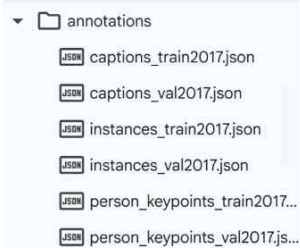
```
!wget http://images.cocodataset.org/annotations/annotations_trainval2017.zip
```

```
!unzip annotations_trainval2017.zip
```

```
# 이미지 데이터셋 다운로드 받기 - /content/val2017 폴더 생성됨
```

```
!wget http://images.cocodataset.org/zips/val2017.zip
```

```
!unzip val2017.zip -d /content/
```



파일명	용도	포함 정보
captions_train2017.json	이미지 캡션	이미지 ↔ 문장 설명
captions_val2017.json	이미지 캡션 검증	동일
instances_train2017.json	객체검출·세그멘테이션	bbox, segmentation, class
instances_val2017.json	검증용	동일
person_keypoints_train2017.json	사람 포즈 추정	17 keypoints 좌표
person_keypoints_val2017.json	검증용	동일

✓ captions_train2017.json / captions_val2017.json

- 이미지 설명 텍스트(캡션) 데이터
 - 예: "A dog running in the park"
 - 자연어 캡셔닝(NLP + CV) 모델 학습용
 - 이미지 ↔ 문장 매핑 정보 포함
-

✓ instances_train2017.json / instances_val2017.json

COCO에서 가장 중요한 파일.

- 객체 검출(Object Detection)
- 인스턴스 세그멘테이션(Mask)
- 카테고리 정보 (80 classes)
- Bounding Box (x,y,w,h)
- Segmentation polygon(세그멘테이션 마스크 좌표)

➡ YOLO, Faster R-CNN, Mask R-CNN 등이 이 JSON을 사용함.

✓ person_keypoints_train2017.json / person_keypoints_val2017.json

- 사람 관절(Keypoints) 정보
- 17개 관절 위치 제공 (코, 눈, 어깨, 팔꿈치 등)
- 포즈 추정(Pose Estimation) 학습용

➡ OpenPose, HRNet, YOLO-Pose 등이 이 데이터 사용.

- instances_*.json → detection + segmentation
- person_keypoints_*.json → pose estimation
- captions_*.json → image captioning

```
from pycocotools.coco import COCO
coco = COCO("annotations/instances_val2017.json")
```

```
# pycocotools.coco.COCO
# COCO JSON(annotation) 파일을 읽어서
# → 이미지·객체·마스크 정보를 쉽게 꺼낼 수 있게 해주는 파이썬 클래스.
# COCO 데이터셋을 파이썬에서 다루기 위한 공식 라이브러리.
# polygon → mask 변환
# RLE 압축
# IoU 계산 등
# COCO 논문과 동일한 포맷 제공.

type(coco)
```



기능	예
COCO annotation JSON 로드	instances_train2017.json 읽기
이미지 목록 가져오기	getImgIds()
특정 이미지 정보 불러오기	loadImgs()
annotation ID 가져오기	getAnnIds()
annotation 정보 불러오기	loadAnns()
polygon → mask 변환	annToMask()
카테고리 정보 읽기	loadCats()

- 완성코드 미리 보기

```

1 image_id = 397133
2 file_name = f"/content/val2017/{image_id:012d}.jpg"
3 print(file_name)
4
5 import cv2
6 import matplotlib.pyplot as plt
7 img = cv2.imread(file_name)
8 img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
9
10 plt.figure(figsize=(3,3))
11 plt.imshow(img)

```

/content/val2017/000000397133.jpg
<matplotlib.image.AxesImage at 0x7b82c1da3230>



```

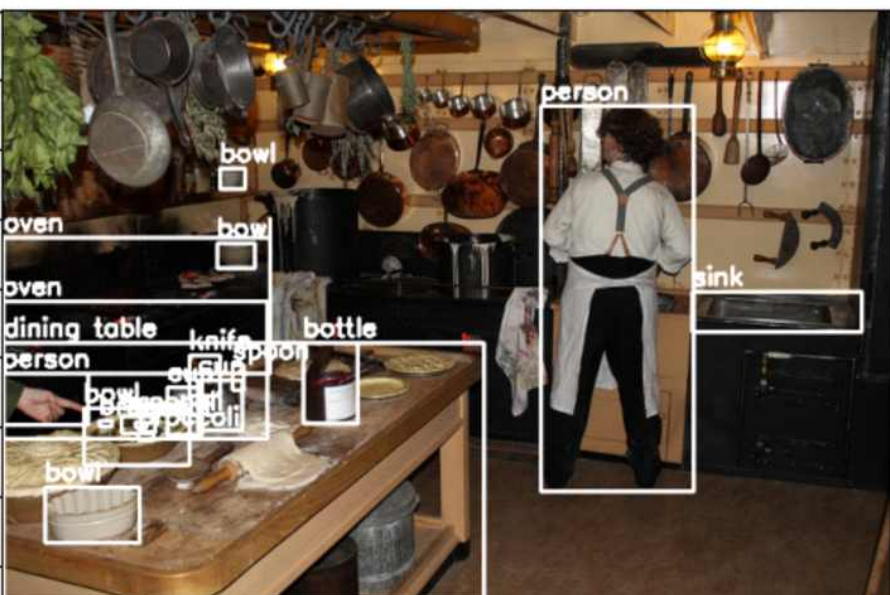
1 from pycocotools.coco import COCO
2
3 # 1) COCO annotation 파일 로드
4 ann_path = "/content/annotations/instances_val2017.json"
5 coco = COCO(ann_path)
6
7 # 2) 조회하고 싶은 이미지 ID
8 image_id = 397133 # 원하는 id로 바꿔도 됨
9
10 # 3) 해당 이미지의 annotation ID 목록 가져오기
11 ann_ids = coco.getAnnIds(imgIds=image_id)
12
13 # annotation 리스트 불러오기
14 anns = coco.loadAnns(ann_ids)
15
16 print(f"\n=== 이미지 ID: {image_id} 의 어노테이션 개수: {len(anns)}개 ===\n")
17
18 # 4) 각 annotation마다 bbox + cat_id + cat_name 출력
19 for ann in anns:
20     bbox = ann["bbox"]
21     cat_id = ann["category_id"]
22     cat_name = coco.loadCats(cat_id)[0]["name"]
23
24     print(f"annotation_id : {ann['id']}")
25     print(f"bbox          : {bbox}")
26     print(f"category_id      : {cat_id}")
27     print(f"category_name    : {cat_name}")
28     print("-----")
29

```

```

1 for ann in anns:
2     x, y, w, h = ann["bbox"]
3     x1, y1, x2, y2 = int(x), int(y), int(x+w), int(y+h)
4     cat_name = coco.loadCats(ann["category_id"])[0]["name"]
5
6     cv2.rectangle(img, (x1, y1), (x2, y2), (255,255,255), 2): W
7     cv2.putText(img, cat_name, (x1, y1-5), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255,255,255), 2)
8
9
10 plt.imshow(img)

```



제공되는 segmentation 값으로 출력하기

```
1 # 3) annotation 중 person(category_id=1)만 추출
2 ann_ids = coco.getAnnIds(imgIds=image_id, catIds=[1], iscrowd=None)
3 anns = coco.loadAnns(ann_ids)
4
5 # 4) 이미지 로드
6 img_info = coco.loadImgs(image_id)[0]
7 img_path = f"/content/val2017/{img_info['file_name']}"
8 img = cv2.imread(img_path)
9 img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
10
11
12
13 # 5) segmentation 값으로 출력
14 plt.figure(figsize=(8,8))
15 plt.imshow(img_points)
16 plt.axis("off")
17 plt.show()
```



이 세그멘테이션 값의 안쪽영역에는 숫자 1로 체크된 mask영역을 빨간색으로 채우기

```
1 # 3) annotation 중 person(category_id=1)만 추출
2 ann_ids = coco.getAnnIds(imgIds=image_id, catIds=[1], iscrowd=None)
3 anns = coco.loadAnns(ann_ids)
4
5 # 4) 이미지 로드
6 img_info = coco.loadImgs(image_id)[0]
7 img_path = f"/content/val2017/{img_info['file_name']}"
8 img = cv2.imread(img_path)
9 img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
10
11 # 5) segmentation 그리기
12 mask_total = np.zeros((img.shape[0], img.shape[1]), dtype=np.uint8)
13
14 for ann in anns:
15     mask = coco.annToMask(ann)
16     mask_total = np.maximum(mask_total, mask)
17
18 # 6) 마스크를 빨간색으로 덧칠
19 img_seg = img.copy()
20 img_seg[mask_total == 1] = [255, 0, 0] # 빨강 색칠
21
22 # 7) 결과 출력
23 plt.figure(figsize=(8,8))
24 plt.imshow(img_seg)
25 plt.axis("off")
26 plt.show()
```



- 이미지 5000장, 각 이미지의 id번호

```
1 len(coco.getImgIds()), coco.getImgIds()[:3]
(5000, [397133, 37777, 252219])
```

COCO는 이미지 파일명을 다음처럼 넣어둬:

e.g. /val2017/000000397133.jpg

여기서

000000397133 → **397133**이 이미지 ID

즉, 이미지 ID는 파일명 끝 숫자와 동일한 고유 번호

val2017

- 000000000139.jpg
- 000000000285.jpg
- 000000000632.jpg
- 000000000724.jpg
- 000000000776.jpg

```
1 #-----
2 ## 397133 id 이미지 불러오기
3 #-----
4 image_id = 397133
5 file_name = f"/content/val2017/{image_id:012d}.jpg"
6 print(file_name)
7
8 import cv2
9 import matplotlib.pyplot as plt
10 img = cv2.imread(file_name)
11 img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
12 plt.imshow(img)
```



```
1 img_id=coco.getImgIds()[0]
2 img_id
```

397133

클릭 또는 Enter 키를 눌러 수정

```
1 ann_ids=coco.getAnnIds(img_ids=img_id)
2 len(ann_ids), ann_ids
```

(19, [82445, 119568, 200887, 693231, 713388, 716434, 1125079, 1218137, 1878837, 1883614, 1902250, 1902971, 1914453, 2105658, 2114911, 2114949, 2139366, 2188144, 2196309])

19개의 어노테이션 즉 19개의 객체가 있음

```
1 anns = coco.loadAnns(ann_ids)
2 anns[0].keys()
3
```

dict_keys(['segmentation', 'area', 'iscrowd', 'image_id', 'bbox', 'category_id', 'id'])

```
1 anns[0]
```

{'segmentation': [[224.24, 297.18, 228.29, 297.18, 234.91, 298.29, 243.0, 297.55, 249.25, 296.45, 252.19, 294.98, 256.61, 292.4, 254.4, 264.08, 251.83, 262.61, 241.53, 260.04, 297.55]], 'area': 1481.3806499999994, 'iscrowd': 0, 'image_id': 397133, 'bbox': [217.62, 240.54, 38.99, 57.75], 'category_id': 44, 'id': 82445}

```
1 anns = coco.loadAnns(ann_ids)
2 ann_category=[ann['category_id'] for ann in anns]
3 print(ann_category) #총 19개의 카테고리
4
```

```
[44, 67, 1, 49, 51, 51, 79, 1, 47, 47, 51, 51, 56, 50, 56, 56, 79, 57, 81]
```

```
1 # 중복값 제거
2 print(np.unique(ann_category, return_counts=True))
```

```
(array([ 1, 44, 47, 49, 50, 51, 56, 57, 67, 79, 81]), array([2, 1, 2, 1, 1, 4, 3, 1, 1, 2, 1]))
```

```
1 categoryList=set(ann_category)
2 print(categoryList)
3 for x in categoryList:
4     print(x, coco.loadCats(x)[0]['name'])
```

```
{1, 67, 44, 79, 47, 49, 50, 51, 81, 56, 57}
```

```
1 person
```

```
67 dining table
```

```
44 bottle
```

```
79 oven
```

```
47 cup
```

```
49 knife
```

```
50 spoon
```

```
51 bowl
```

```
81 sink
```

```
56 broccoli
```

```
57 carrot
```

(2) PASCAL VOC

- **특징:**
 - 20개 클래스, 약 11,000장 이미지.
 - Semantic Segmentation과 객체 탐지를 지원.
- **적용 사례:** 도로 위 차량 탐지, 간단한 객체 인식 연구.

```
<annotation>
  <object>
    <name>person</name>
    <bndbox>
      <xmin>48</xmin>
      <ymin>240</ymin>
      <xmax>195</xmax>
      <ymax>371</ymax>
    </bndbox>
  </object>
</annotation>
```

항목	COCO	VOC
포맷	JSON	XML
segmentation	polygon (x,y 리스트) 또는 RLE	binary mask(폴더에 PNG), 또는 polygon 없음
bbox	[x, y, w, h]	<bndbox> <xmin>...
여러 polygon	지원함	보통 지원 안함
mask	annToMask() 바로 사용	별도 PNG로 저장되어 있음

(3) Cityscapes

- **특징:**
 - 자율 주행을 위한 도시 거리 이미지.
 - 19개 클래스, 약 5,000장 이미지.
- **적용 사례:** 도로, 보행자, 차량, 표지판 등 자율 주행 환경 세그멘테이션.

(4) ADE20K

- **특징:**
 - 150개 클래스, 약 20,000장 이미지.
 - 복합 환경 세그멘테이션 (실내, 실외 모두 포함).
- **적용 사례:** 복잡한 장면 분석, 고밀도 환경 연구.

(5) Medical Segmentation Decathlon

- **특징:**
 - 다양한 의료 영상(CT, MRI 등)에서 10가지 태스크 제공.
 - 데이터 크기 및 해상도가 매우 높음.
- **적용 사례:** 의료 영상 분석 연구 (간 종양, 심장 MRI 등).

(6) DeepGlobe Land Cover

- **특징:**
 - 위성 이미지에서 토지 유형 분류.
 - 농지, 도시, 강, 숲 등의 클래스 제공.
- **적용 사례:** 환경 보호, 도시 계획, 재난 관리.

○ 데이터셋 선택 기준

- **태스크에 맞는 데이터셋 선택:**
 - **Semantic Segmentation:** COCO, ADE20K, Cityscapes.
 - **Instance Segmentation:** COCO, PASCAL VOC.
 - **특정 도메인:**
 - **의료:** Medical Segmentation Decathlon.
 - **위성:** DeepGlobe Land Cover.
- **데이터셋 크기:**
 - 대규모 데이터셋: 모델의 일반화 성능을 높임.
 - 소규모 데이터셋: 간단한 실험 및 프로토타이핑에 적합.

4 전이학습 & Segmentation

전이학습 Backbones (Encoder)

세그멘테이션 모델의 전이학습은 대부분 Encoder(backbone) 가 pretrained임.

대표 backbone:

- ResNet (18/34/50/101)
- ResNeXt
- EfficientNet B0~B7
- MobileNet v2/v3
- Swin-Transformer
- ViT-B/L/H
- Xception

이 Backbone이 ImageNet/IN22K에서 사전학습되어 Segmentation 모델에 그대로 끼워넣는 구조임

• FCN 계열 (기본 구조)

모델	특징	전이학습 여부
FCN-32s / 16s / 8s	가장 기본, Segmentation의 시작	ImageNet 기반 encoder 전이학습 가능
U-Net (2015)	의료영상에서 시작, skip-connection 강력	다양한 encoder(backbone)로 전이학습 가능
U-Net++(nested)	U-Net 개선형, 더 촘촘한 skip	많은 pretrained 제공
ResUNet / UNet-ResNet	Encoder를 ResNet으로 교체	ResNet pretrained 사용

• Encoder-Decoder 강자

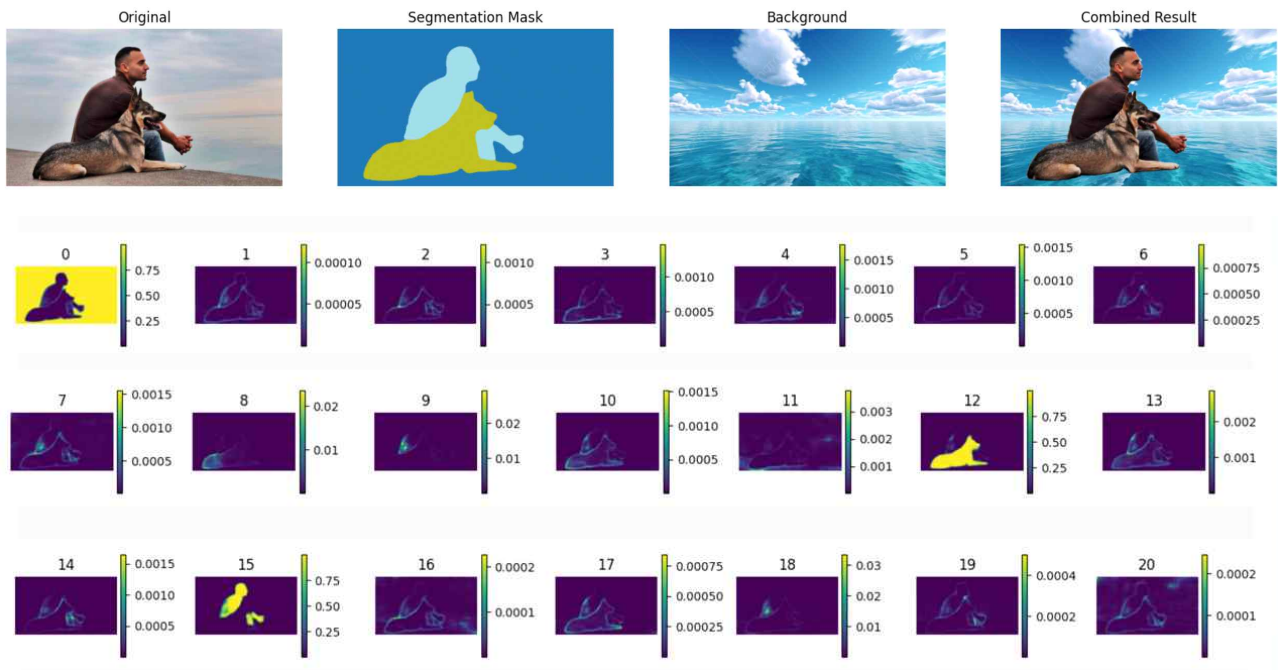
모델	핵심	전이학습
SegNet	pooling index 사용 → 메모리 효율	ImageNet pretrained encoder
DeepLab v1~v3+	Atrous CNN + ASPP, SOTA급	ResNet/Xception backbone pretrained
PSPNet	Pyramid Pooling으로 글로벌 문맥	ResNet backbone pretrained
HRNet-Seg	고해상도 유지, 경계 우수	ImageNet pretrained
BiSeNet v1/v2	실시간 모바일용, 빠름	일부 pretrained 제공

○ 직접 정리합니다.

- Transformer 기반 (Vision Transformer 계열)
- Instance Segmentation 기반 모델 (Mask R-CNN 계열)
- 실시간/모바일용 Segmentation

0.전이학습_Segmentation. ipynb 실습

결과미리보기



- 사전학습 모델

```

1 from torchvision.models.segmentation import DeepLabV3_ResNet50_Weights
2
3 # 가중치 불러오기
4 weights = DeepLabV3_ResNet50_Weights.DEFAULT
5 meta = weights.meta
6 print(meta.keys())
7 print(len(meta["categories"])) # 학습된 클래스 레이블 목록
8
dict_keys(['categories', 'min_size', '_docs', 'num_params', 'recipe', '_metrics', '_ops', '_file_size'])
21
1 # -----
2 # DeepLabV3_ResNet50_Weights.DEFAULT 는 PASCAL VOC 2012 데이터셋으로 훈련
3 # VOC segmentation 클래스는 총 21개
4 # 20개 object class + 1개 background class
5 # -----
6 print(len(meta['categories']))
7 display(meta["categories"])

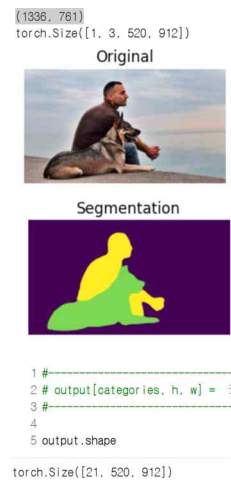
```

- 사전학습모델을 이용한 세그멘테이션후 output값 확인

```

7 from torchvision import transforms
8 from PIL import Image
9 import matplotlib.pyplot as plt
10
11 # 1. 모델 불러오기
12 # DeepLabV3-ResNet50 모델용 공식 사전학습 weight 세트 중에서 DEFAULT를 쓰고자
13 weights = DeepLabV3_ResNet50_Weights.DEFAULT
14 model = deeplabv3_resnet50(weights=weights)
15 model.eval()
16
17 # 2. 전처리 파이프라인 (weights에 내장된 transform 사용 가능)
18 preprocess = weights.transforms()
19
20 # 3. 이미지 불러오기
21 img = Image.open("/content/test_seg.jpg")
22 print(img.size) # (1336, 761)
23 x = preprocess(img).unsqueeze(0) # torch.Size([1, 3, 520, 912])
24 print(x.shape)
25
26 # 4. 추론
27 with torch.no_grad():
28     output = model(x)["out"][0] # [num_classes, H, W]
29 pred = output.argmax(0)

```



```

1 # -----
2 # output[categories, h, w] = 카테고리 점수 (logit)
3 # output[0] -> 배경(배경) 클래스의 logit score map 로서 픽셀별 "그 클래스일 가능성을 나타내는 점수" 임니
4 # output[1] -> aeroplane(비행기)의 logit
5 # -----
6 output.shape

torch.Size([21, 520, 912])

1 # -----
2 # 0.0픽셀을 21개의 채널을 모두 다 확인했을때 가장 큰 값은 첫번째 값임
3 # 그래서 background임
4 # -----
5 for x in range(21):
6     print([output[x][0,0]])

tensor(10.1892)
tensor(-2.1442)
tensor(-2.2819)
tensor(-1.6201)
tensor(-0.4136)
tensor(-0.5689)
tensor(-1.0188)
tensor(1.6265)

```

실습 1 — 이미지에서 특정 객체 제거 (Inpainting 기반)

목표: 사람영역을 세그멘테이션으로 찾은 뒤, 그 영역을 OpenCV inpaint로 “지워서” 배경만 남기기.

✓ 활용 예: 얼굴 가리기, 개인정보 보호, 객체 제거

실습 2 — 배경만 블러 처리 (Portrait Mode)

목표: 사람만 선명하게 / 배경에만 Gaussian Blur 적용

✓ 활용 예: 스마트폰 인물모드 / 머신비전에서 관심 영역(ROI) 강조

실습 3 — 세그멘테이션 기반 경계선(edge) 추출

목표: 사람 mask에서 openCV Canny + findContours 로 윤곽선 따기

✓ 제조 라인 불량 형태 분석용으로 많이 사용함

Mask 15



Canny



Contours

